Faster workflows. Measured trade-offs.

1.2.2TF / Bert1 / 1.2.2xTF

DEEPSEEK V4.1 FLASH • GB10 / SPARK-CLASS TP3

A matched-input comparison of tool-using workflows and an online learner for speculative verification budgets.

234	8.76%	30 → 16
completed tasks	shorter summed waves[1]	rejected tool attempts[1]

[1] xTF compared with 1.2.2TF in one pass. These are observed workflow results; the test does not isolate the cause of the gains.

What the comparison actually found

78 completed tasks per profile, C1–C12, one matched-initial-input pass. Generated output includes reasoning.

Measured outcome	1.2.2TF	Bert1	1.2.2xTF
Completed / attempted tasks	78 / 78	78 / 78	78 / 78
Sum of measured wave times (m:ss)	22:27.872	23:38.938	20:29.795
Task duration: median / P95 (s)	117.05 / 170.94	118.05 / 182.52	108.73 / 154.03
Longest completed task (s)	190.31	239.52	159.45
Model requests	350	350	331
Generated tokens, including reasoning	164,239	167,522	153,259
Rejected intermediate tool attempts	30	31	16
Tasks with at least one rejection	20 / 78	20 / 78	13 / 78
Draft acceptance, pooled	79.13%	77.52%	83.32%
Weighted uncached prefill (tokens/s)	847.64	841.00	860.99

xTF: 8.76% less summed wave time than TF; 13.33% less than Bert1. It also generated 6.69% fewer tokens than TF, so the full time saving cannot be assigned to decode speed alone.

Three recipes, several moving parts

The comparison changes a serving recipe, rather than isolating a single knob.

Configuration	1.2.2TF	Bert1	1.2.2xTF
Native stream / serving slot cap	C12	C16	C12
Shared verification-row cap	R32	R48	R48
KV pool capacity (tokens)	8,000,000	7,000,000	7,000,000
Prompt snapshot-cache budget	2 GiB	2 GiB	3 GiB
Gateway token ceiling (not exercised)	5,336,064	6,699,008	5,336,064
Speculative allocation	Original cost-based	Original cost-based	Online calibrated
Timing / acceptance learner	Off	Off	On, process-local

Held constant

DeepSeek V4.1 Flash with the same frozen quantized weights, tokenizer and Engram assets; TensorFold native CUDA stack; three GB10 / Spark-class systems in TP=3; GPU clock 2,200 MHz; existing graph, prefetch, RDMA and sampling paths. MAX reasoning for these text workflows.

Vision was unchanged: C2 / MEDIUM. No vision images or mixed text–vision load were benchmarked here. Original TF and Bert1 already allocated drafts dynamically using cost priors.

A small agent workflow, end to end

A real local tool loop, not a fixed-length text-generation microbenchmark.

1

01 Repair code

Fix merge_events: newest timestamp per request ID; later input wins ties; sort by ID; preserve the input and return copies.

2

02 Execute tests

Use the isolated run_tests tool. Its safe function evaluator forbids imports; invalid operations are returned to the model as errors.

3

03 Do accounting

Read two pages of records, deduplicate by ID and latest timestamp, keep ds41 / done records, and total requests plus token usage.

4

04 Finish clearly

Submit correct results and 4–6 Czech explanatory sentences (30–180 words). The report is English; the tested task language was Czech.

Tools: **read_file, replace_file, run_tests, query_records, finish_task.** Deterministic grading checks code, tests, accounting and output format; it does not fully grade the meaning of the prose.

How the runs were made comparable

Initial inputs match; the model's later actions are free to diverge.

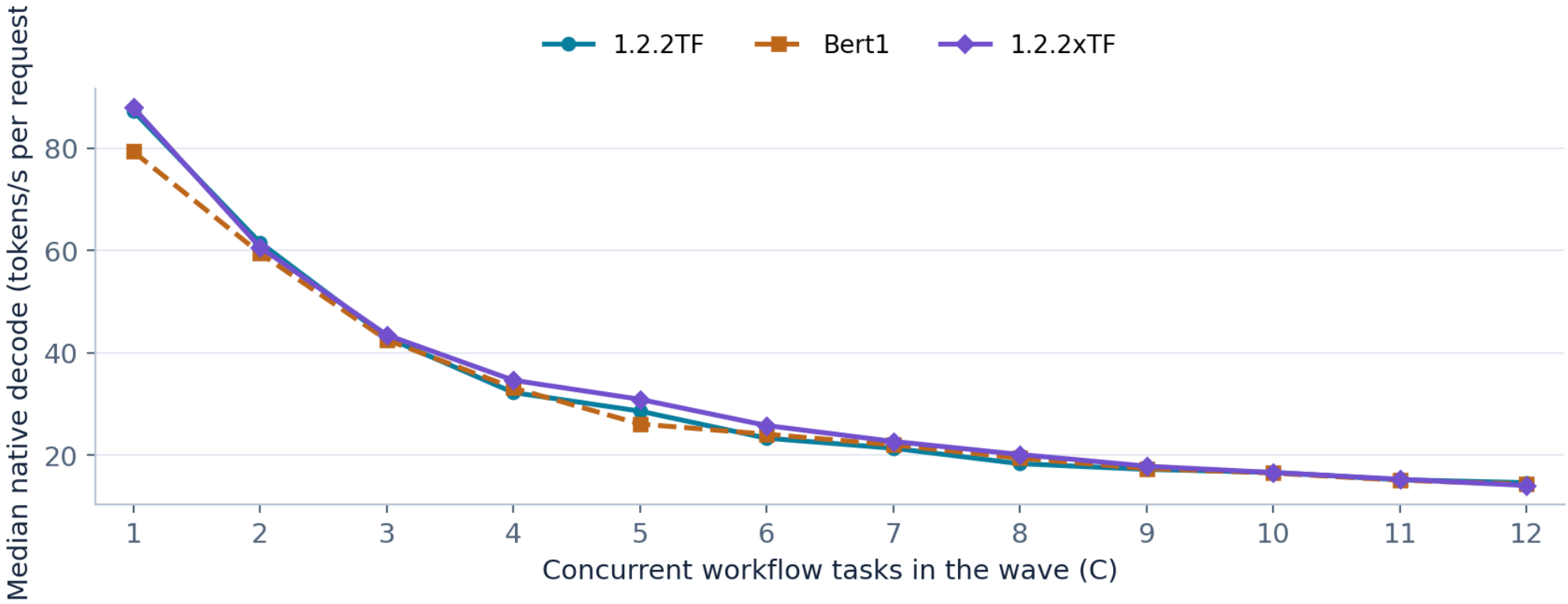
Protocol detail	What was used
Load sweep	C1–C12, with C simultaneous workflow tasks per wave
Work per profile	1 + 2 + … + 12 = 78 tasks; 12 measured waves
Sampling	Temperature 0.6; seed 42; MAX reasoning
Completion policy	Natural task completion; max 65,536 tokens/call; 16-turn guard
Input matching	Corresponding initial request hashes verified across all 36 cells
Timing path	Direct native API via an owned tunnel; public ingress held closed
Failure accounting	Rejected tools and repair turns remain in time and error totals

Measured work: TF 22:27.872 + Bert1 23:38.938 + xTF 20:29.795 = **66:36.605** summed wave time. This excludes startup, profile switching, waiting and an excluded partial attempt; it is not continuous campaign wall time.

No repeated trials, randomised profile order or confidence intervals. Bert1 used an already-running process; TF and xTF used restarted processes. Cache warmness was therefore not identical.

Decode improved most in the middle of the sweep

Native per-request median decode TPS. These are not aggregate GPU tokens per second.



+8.04%
C5 vs TF

+10.63%
C6 vs TF

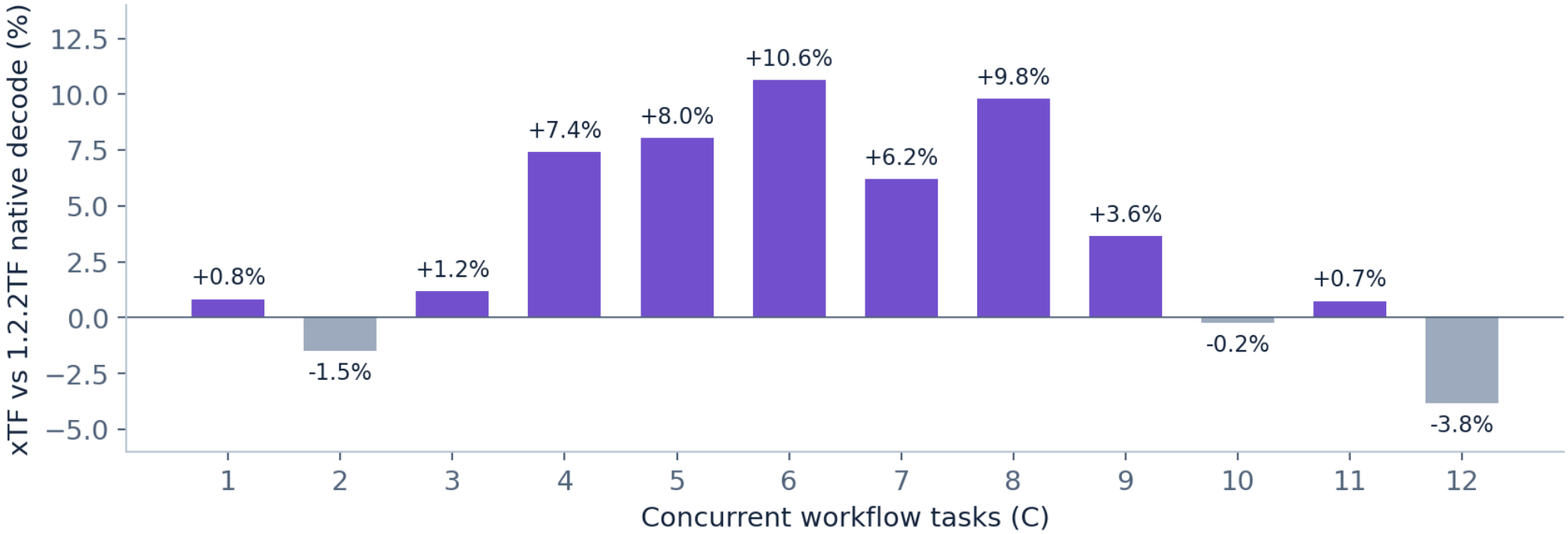
+6.20%
C7 vs TF

−3.81%
C12 vs TF

xTF did **not** obtain fewer tool errors by slowing native decode at C6/C7: its medians were faster there. At C12, native decode was slightly slower than TF.

The gains were uneven across concurrency

Same native metric, normalised against 1.2.2TF. No error bars are claimed from a single pass.

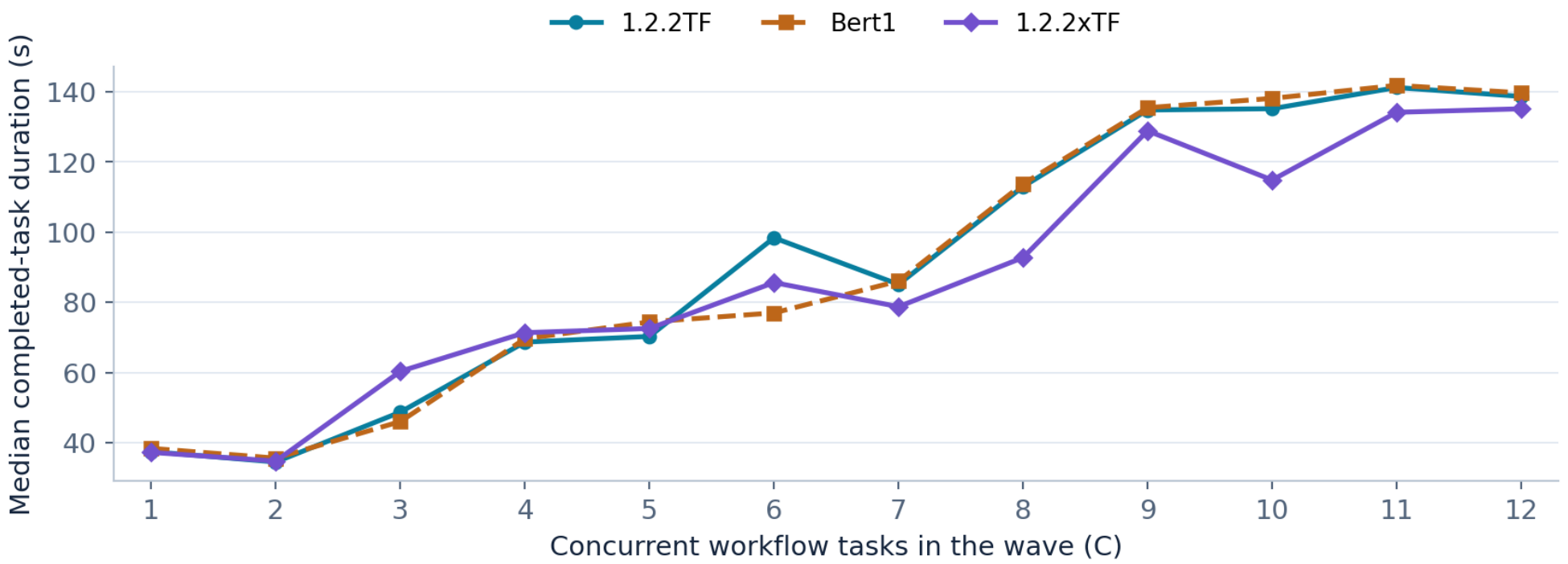


The R48 ceiling is a limit the allocator can use, not a demand to run 48 rows every round. xTF kept approximately the original C1 performance, improved several middle-load points, and did not win every cell.

Workflow C counts parallel agent tasks. Each task pauses for tools and makes multiple requests; actual active native streams can be lower and change within a wave.

Task completion time is the practical metric

Full task duration includes reasoning, tools, rejected attempts and repair turns.



Counterexample	1.2.2TF	Bert1	1.2.2xTF
C3: median task seconds	48.69	46.00	60.31
C6: median task seconds	98.38	76.97	85.62
C12: median task seconds	138.68	139.70	135.16

At C3, xTF generated 8,286 tokens vs TF's 5,602 despite slightly faster decode. At C6 it decoded faster than Bert1 but took longer. At C12 it finished sooner despite lower native TPS.

Prefill, TTFT and the first post-prefill token

Native timing distributions pooled across all requests. Units are milliseconds except the final row.

Pooled native measurement	1.2.2TF	Bert1	1.2.2xTF
Prefill: median / P95 / max	235.49 / 864.06 / 1,078.16	232.88 / 871.53 / 902.96	235.87 / 866.33 / 1,068.42
Received → first token: median / P95 / max	356.80 / 5,990.96 / 10,220.80	372.30 / 6,033.55 / 10,300.57	382.51 / 5,993.82 / 10,224.85
Post-prefill upper bound: median / P95 / max	4.14 / 7.17 / 8.97	3.91 / 7.33 / 16.00	3.72 / 7.12 / 8.90
Uncached prefill: weighted tokens/s	847.64	841.00	860.99

The 3.72 ms xTF number is an upper bound

The frozen native runtime does not export the exact post-prefill event interval. We derive an upper bound that removes scheduling wait and prefill, but still includes preparation before submission. Native TTFT includes more work; its xTF median was 382.51 ms, not 3.72 ms.

```
Upper bound = native TTFT − native request seconds + (completion tokens − 1) / native decode TPS
```

Client first-delta latency is also exported per C. It can refer to a reasoning/tool delta, rather than the first visible answer token. Exact post-prefill latency remains null in request-timings.csv.

Fewer rejected tools; no failed final tasks

Intermediate tool rejections were retained and repaired. They are not API crashes or final-task failure counts.

Observed rejection category	1.2.2TF	Bert1	1.2.2xTF
Unavailable test-file reads / invalid arguments	11	10	2
Incorrect request accounting	6	6	2
Forbidden imports (Import + ImportFrom)	11	11	10
Other function-shape / output-format errors	2	4	2
Total rejected attempts	30	31	16
All tool attempts (denominator)	508	508	487
Tasks affected / completed	20 / 78	20 / 78	13 / 78

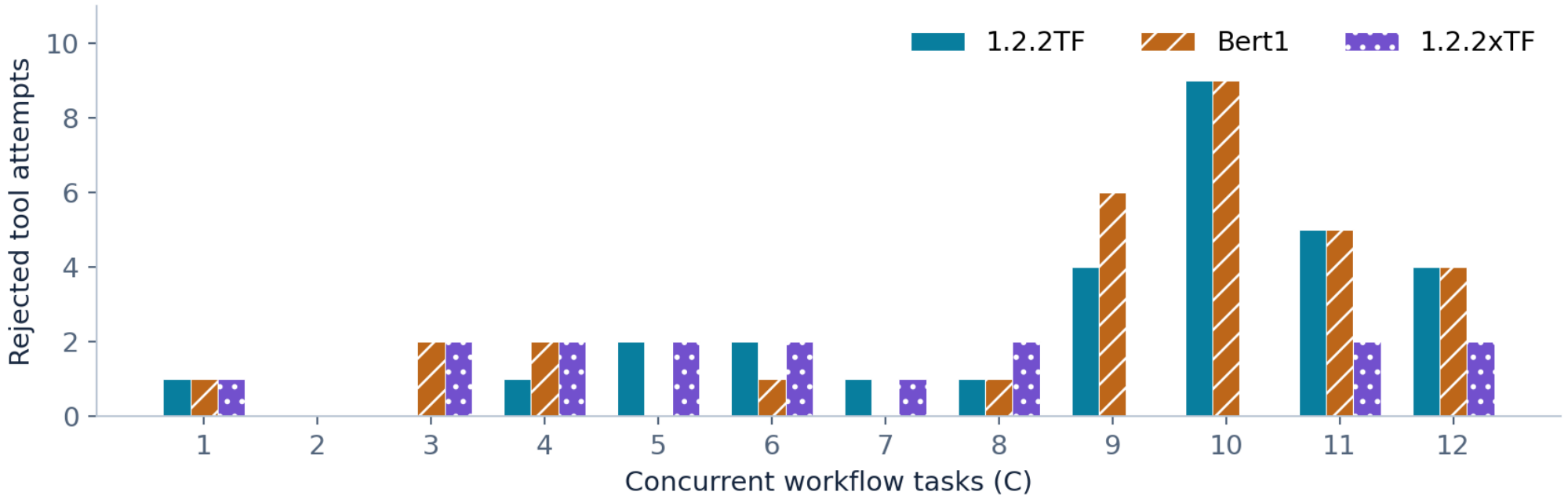
What improved in this sample

Most of the difference came from fewer attempts to read unavailable test files and fewer incorrect accounting submissions. The import restriction still caused 10 xTF rejections. All profiles ultimately passed 78/78 tasks.

> 30 → 16 is a **46.67% reduction in counted intermediate rejections**, not a demonstrated 46.67% improvement in general reasoning or production reliability.

The error reduction was concentrated

One pass can be strongly affected by a small number of diverging tool trajectories.



Concurrency band	1.2.2TF	Bert1	1.2.2xTF
C1–C8 rejected attempts	8	7	12
C9–C10 rejected attempts	13	15	0
C11–C12 rejected attempts	9	9	4

xTF had more rejections in C1–C8 and fewer in C9–C12. This is useful evidence for real-workflow evaluation, but does not show that larger concurrency inherently causes errors, or identify a cache/allocator mechanism.

Memory headroom and prompt-cache reuse

KV capacity and reusable prompt snapshots are different resources.

Observed memory / cache metric	1.2.2TF	Bert1	1.2.2xTF
Minimum sampled MemAvailable (GiB)	7.227	4.091	6.190
Maximum memory full PSI avg10 (%)	0.00	0.05	0.00
Native cancellations / preemptions	0 / 0	0 / 0	0 / 0
Prefix token reuse fraction	83.16%	83.48%	81.46%
Actually computed prompt tokens	124,409	123,864	122,660

What 3 GiB changes

A larger snapshot budget can keep more reusable prompt state and reduce recomputation when that state is reused. Eviction is not evidence that the model lost text supplied in the next request. xTF's token reuse fraction was actually lower here (81.46% vs 83.16% TF), so this run does not establish a cache-driven speedup or quality gain. [4]

Coverage caveat: Bert1's retained per-rank monitor files cover the continuation window, whereas TF/xTF monitoring covers their measured sweeps. The smaller Bert1 minimum is descriptive, not a controlled memory comparison. xTF's sampled full-memory PSI remained zero.

What the adaptive allocator actually controls

It learns which verified draft rows are worth their runtime cost. It does not train the model's weights.

Draft candidates
Existing head/copy lanes propose candidate chains

→

Estimate utility
Draft survival + predicted verification cost

→

Choose a budget
Keep legal prefixes, within R48 shared cap

→

Target verification
Existing verifier decides which tokens are accepted

Timing + reached-prefix acceptance feedback → update process-local estimates

R is shared, not a per-request draft length

R48 caps total verification rows in one shared round, including mandatory anchor rows. With six active streams, the allocator does not simply assign eight speculative tokens each. Candidate confidence, context, timing and legal prefix dependencies determine the allocation.

> The target verifier checks token agreement, not whether a tool command is semantically valid. Better draft acceptance can improve efficiency; it does not inherently make the target model smarter.

Learning the runtime price of a row

Original cost-based allocation is the starting point. Online feedback and average-reward pricing are inspired by AdaSpark [2].

Condition on the work being done

Cost buckets use sampling mode, active-stream bands (1, 2, 3–4, 5–6, 7–8, 9–16), context bands (≤8K through >256K), and row width. Context is a mandatory-row-weighted mean, so selecting more drafts does not redefine the input feature.

Choose expected output per unit of time

Cold start: maximise expected emitted tokens / predicted round time. After 16 reward observations in a bucket, use an average-reward price ρ = smoothed emitted tokens / smoothed elapsed time, and score expected tokens − ρ × predicted time.

A conservative local timing fit

Each width keeps the latest 16 measured/base timing ratios and uses their median. The first two sightings are skipped for cold shape/graph effects. Sparse widths borrow a correction only within the same load/context/mode bucket; trust grows as n / (n + 8).

Cost prior: original R1–R32 table (R25–R32 were already extrapolated); R33–R48 extended by +3.1 ms/row. These extensions are priors, not independently calibrated offline measurements.

Implementation guards: correction clipped to 0.5–3×; timing samples outside 0.25–8× baseline rejected; overhead EWMA α=0.1; reward/time EWMA α=0.025; 2% incumbent margin; nearby useful probes at most every 64 rounds, within a 4% utility margin.

Learning draft survival, without inventing failures

Calibration is separate from target-token verification and from model-weight training.

What gets a label

For each reached draft position, record whether it matches the target choice. After the first mismatch, later draft positions were not reached: they are censored, rather than all being counted as failures. Buckets distinguish sampling mode, head/copy source, depth (16 bins) and prior confidence (8 bins).

```
q = (32 × prior + successes × retain) / (32 + trials × retain)
retain = 1 / (1 + age_in_rounds / 512)
```

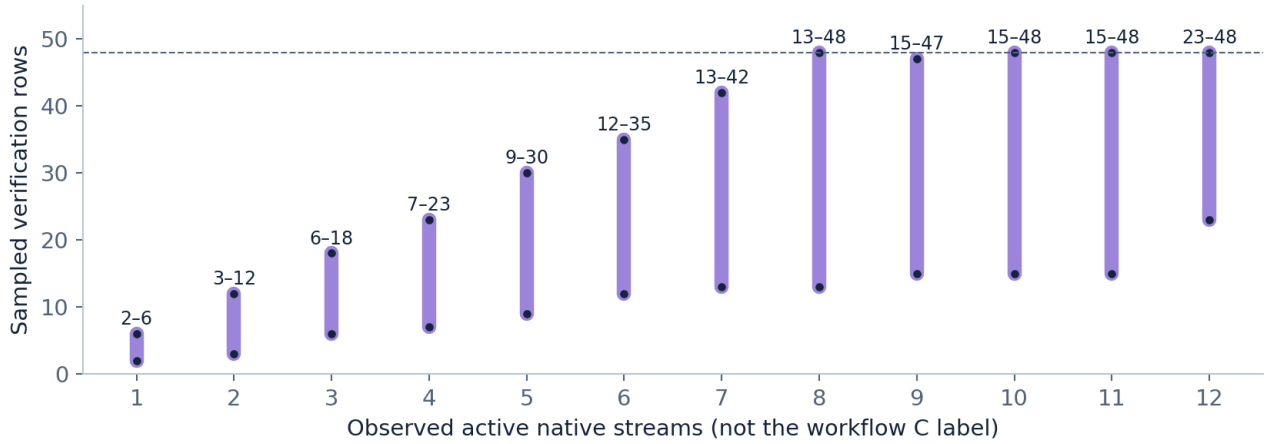
Keep adaptation reversible

Counts halve at 128 trials; stale unobserved buckets gradually move back toward the current prior. Learned calibration is used only after at least 32 guard observations and while its EWMA log loss is no worse than 1.01× the raw prior. Path survival is the product of conditional probabilities; parents precede children.

> State is shared across requests in the native process. A new chat does not reset it; a process restart does. There is no disk persistence and no guarantee of monotonic improvement.

The learner was active during the xTF run

These are measured counters and sampled ranges, not a reconstructed convergence curve.



11,672
Controller rounds

11,267
Cost updates

112,076
Acceptance labels

2
Nearby-width probes

0
Rejected timing samples

Ranges use one telemetry sample every 3 seconds; they are not every-round minima/maxima. A C12 workflow can briefly have only one active native stream. Observed widths varied with load, rather than remaining at R48.

Timing fit: Σ|predicted − actual verify time| / Σactual verify time = **4.76%**, over 11,267 qualified prediction samples. Mean absolute error was **4.10 ms**; measured verify time summed to 969.52 s. This is neither MAPE nor “95.24% prediction accuracy”.

What explains the changed behaviour?

Separate mechanisms we know exist from causes this test did not identify.

Finding	Interpretation
Higher pooled acceptance: 83.32% xTF	Observed; +4.19 percentage points vs TF. Load/width and draft selection also changed.
Shorter summed workflow time	Observed; faster middle-load decode plus fewer tokens/calls/repairs contributed.
Lower intermediate tool-error count	Observed; concentrated at C9/C10. One pass does not isolate a semantic quality effect.
Larger snapshot cache	Can reduce recomputation. This run’s token reuse fraction did not increase.
Same seed, different trajectories	Same initial inputs do not guarantee identical generated paths under different execution shapes.

The frozen sampler keys randomness by seed, absolute position and token ID; blaming a changed RNG-consumption order would be unsupported. GPU batch shapes can change floating-point results in general, but no token-level numerical diagnosis was performed here. [3]

> The useful conclusion is an improved measured recipe for this workflow. The test does not show that reduced KV capacity or a larger prompt cache fixes long-session repetition loops.

Run integrity and what remains untested

The report preserves operational exceptions instead of turning them into model-quality claims.

One excluded attempt, transparently retained

An initial Bert1 C10 attempt was stopped by a cold swap-occupancy guard after roughly 304 MiB of growth, without evidence of real memory pressure. C1–C9 were reused only after strict initial-input hash checks. C10–C12 were rerun with a shared fresh continuation namespace across all three profiles. The partial attempt is excluded from the tables, not erased.

Final API reopening was a separate operational step

The first post-test API reopening did not complete and the controller rolled back automatically. Later promotion reopened and verified the API. Neither event is counted as a workflow tool error or model task failure; deployment/startup time is excluded from benchmark durations.

Not established by this benchmark

Long-session repetition resistance; 7M/8M pool saturation or full advertised context capacity; vision performance or mixed image/text contention; hours-long production reliability; model quality on other tasks; independent contributions from R48, cache size and online calibration.

> The same candidate had 322 native CPU tests pass before the GPU workflow run. Passing unit tests and passing this workflow are separate forms of evidence, not a general quality certification.

A1 / Full workflow durations

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Wave elapsed (s)			Median task (s)			P95 task (s)		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	37.42	38.34	37.20	37.42	38.34	37.20	37.42	38.34	37.20
2	36.48	37.66	36.75	34.46	35.57	34.72	36.48	37.66	36.75
3	51.81	64.66	83.19	48.69	46.00	60.31	51.80	64.66	83.19
4	84.83	83.22	79.59	68.65	69.59	71.33	84.81	83.22	79.59
5	86.28	81.72	95.75	70.27	74.41	72.53	86.28	81.72	95.75
6	115.14	96.33	120.53	98.38	76.97	85.62	115.12	96.31	120.53
7	99.05	87.66	91.34	85.09	86.05	78.73	99.05	87.66	91.34
8	127.38	132.11	107.88	112.97	113.73	92.74	127.38	132.09	107.88
9	167.58	182.52	139.14	134.81	135.47	128.89	167.58	182.52	139.12
10	183.73	239.52	124.92	135.18	138.12	114.84	183.73	239.52	124.92
11	167.83	173.64	154.03	141.23	141.84	134.16	167.81	173.64	154.03
12	190.34	201.58	159.47	138.68	139.70	135.16	190.31	201.56	159.45

Wave elapsed is time until the last task in that C wave finishes. Task distributions describe individual workflows, including tools and repairs. With C1, median and P95 refer to the same single task.

A2 / Native decode distributions

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Median (tokens/s)			P95 (tokens/s)			Weighted (tokens/s)		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	87.28	79.35	88.00	107.88	98.36	108.65	84.37	82.36	84.95
2	61.59	59.51	60.68	80.34	67.92	82.44	56.66	55.13	56.37
3	43.02	42.58	43.53	67.75	89.28	85.91	42.63	44.91	46.65
4	32.27	33.16	34.66	79.80	57.85	92.68	32.93	32.92	34.47
5	28.63	26.11	30.93	72.31	33.17	99.01	27.67	26.82	30.63
6	23.34	24.12	25.82	38.60	53.83	63.17	23.61	25.37	26.23
7	21.38	22.03	22.70	35.42	31.25	36.90	21.66	20.65	22.80
8	18.36	19.44	20.15	38.78	34.67	35.40	19.53	19.93	19.82
9	17.24	17.41	17.87	48.18	48.09	34.01	18.63	18.91	17.62
10	16.66	16.52	16.62	76.11	73.60	27.30	18.14	19.79	16.50
11	15.20	15.11	15.31	31.87	29.38	31.59	15.15	15.10	15.76
12	14.65	14.36	14.09	49.81	53.49	26.91	15.26	15.21	14.31

All three are per-request native decode rates. Weighted = summed generated tokens / summed request decode seconds. Request decode times overlap under concurrency; this is not aggregate device throughput.

A3 / Rates seen by the workflow

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Request median (tok/s)			Whole-wave (tok/s)			Correct tasks/min		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	75.80	73.28	77.92	78.99	77.09	79.46	1.60	1.56	1.61
2	46.17	41.80	47.26	98.73	95.66	98.01	3.29	3.19	3.27
3	38.31	38.43	37.81	108.12	96.70	99.61	3.47	2.78	2.16
4	30.88	28.99	32.70	105.41	103.95	112.84	2.83	2.88	3.02
5	24.52	24.57	26.49	111.46	113.41	112.68	3.48	3.67	3.13
6	21.87	21.30	22.12	114.65	114.93	114.26	3.13	3.74	2.99
7	20.08	19.45	20.12	120.81	125.21	125.52	4.24	4.79	4.60
8	17.90	17.45	17.91	124.41	121.68	128.11	3.77	3.63	4.45
9	16.66	16.60	17.05	128.10	125.17	136.37	3.22	2.96	3.88
10	16.42	15.81	15.28	127.90	113.39	137.39	3.27	2.51	4.80
11	14.94	13.80	14.72	135.22	132.34	137.67	3.93	3.80	4.28
12	14.11	14.01	13.50	130.53	127.43	139.49	3.78	3.57	4.52

Request rate includes queue, prefill and delivery; whole-wave rate includes tool gaps and repairs. Both include reasoning tokens. Neither is interchangeable with pure native decode TPS.

A4 / Prefill durations and weighted speed

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Median duration (ms)			P95 duration (ms)			Weighted speed (tok/s)		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	230.42	264.36	237.73	1,078.16	867.88	1,068.42	748.38	728.79	747.46
2	369.83	399.80	371.43	844.15	847.41	836.99	885.56	840.25	885.89
3	368.76	233.20	220.43	863.25	871.55	856.65	886.54	799.14	855.62
4	329.40	223.85	225.96	866.09	882.48	884.30	844.66	820.54	829.83
5	395.10	380.76	229.09	854.03	868.16	860.01	816.72	867.91	848.93
6	245.32	233.62	232.53	860.59	868.24	857.52	837.44	867.14	860.52
7	222.71	383.62	235.87	863.33	868.04	867.44	870.41	869.22	868.54
8	242.82	248.42	225.87	875.12	864.03	860.63	863.01	864.70	863.53
9	221.70	222.80	495.08	858.54	873.23	884.32	851.59	836.00	863.95
10	225.56	228.14	382.46	869.81	874.43	854.61	832.67	817.98	876.50
11	239.42	231.53	234.69	868.87	871.53	868.04	837.34	836.53	861.64
12	222.89	228.02	239.33	863.11	874.83	865.19	859.95	844.68	864.67

Prefill speed uses actually computed prompt tokens, excluding reused prefix tokens. Short requests and reuse can make per-request prefill rates very different from a large uncached-prompt microbenchmark.

A5 / Uncached prefill work

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Median speed (tok/s)			P95 speed (tok/s)			Computed prompt tokens		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	347.19	302.62	336.52	953.47	1,184.50	962.16	1,601	1,601	1,601
2	561.04	507.30	552.22	1,244.41	1,213.11	1,230.37	3,129	3,129	3,129
3	551.77	338.59	351.96	1,234.73	1,211.55	1,232.72	4,674	4,917	4,718
4	416.38	345.41	354.05	1,215.79	1,199.27	1,217.49	6,471	6,450	6,527
5	440.39	544.00	338.90	1,228.88	1,199.05	1,222.96	8,461	7,788	8,077
6	333.02	347.64	352.93	1,201.95	1,206.67	1,203.97	9,606	9,351	9,406
7	329.70	535.58	338.91	1,200.90	1,190.32	1,202.04	10,896	10,875	10,924
8	340.06	353.42	337.02	1,217.48	1,233.34	1,218.27	12,466	12,471	12,474
9	348.66	336.11	634.24	1,203.32	1,208.36	1,231.67	14,381	14,454	14,305
10	345.11	334.27	546.25	1,212.20	1,204.16	1,221.64	16,005	15,990	15,602
11	349.00	355.85	348.09	1,221.17	1,213.94	1,212.74	17,958	17,955	17,190
12	338.69	335.27	343.75	1,217.83	1,213.93	1,215.57	18,761	18,883	18,707

Median/P95 are distributions of per-request computed-token prefill rates. The weighted speed on A4 uses total computed tokens / total prefill seconds. These statistics need not agree.

A6 / Native and client first output

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Native TTFT median (ms)			Native TTFT P95 (ms)			Client first delta (ms)		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	234.80	268.74	245.31	1,079.86	870.49	1,070.80	500.00	610.00	453.00
2	396.91	499.71	388.27	1,652.86	1,695.80	1,673.81	546.50	703.50	563.00
3	402.15	249.26	244.43	2,518.59	2,544.45	2,519.91	656.00	500.00	312.50
4	387.67	284.01	295.75	3,404.66	3,464.71	3,439.90	570.50	515.00	562.00
5	438.44	453.39	287.80	3,382.66	3,434.83	3,394.81	875.00	969.00	718.00
6	373.59	315.15	347.17	4,255.23	4,285.41	4,235.11	656.00	703.00	632.50
7	350.29	450.54	363.90	5,135.78	5,170.13	5,133.20	641.00	1,047.00	1,047.00
8	459.28	380.54	317.62	5,990.96	5,964.90	5,953.24	672.00	875.00	601.50
9	323.94	374.75	589.68	5,973.60	6,038.59	6,840.73	672.00	797.00	734.00
10	351.85	343.82	538.84	6,874.47	6,851.47	6,759.24	578.00	594.00	1,148.00
11	354.48	429.83	372.74	7,674.90	7,732.63	7,681.16	688.00	765.00	1,163.50
12	395.65	378.61	402.82	8,479.62	8,572.12	8,506.17	1,195.00	999.50	1,258.00

Native TTFT starts at native request receipt. Client first delta starts at client request submission and may be a reasoning or tool delta. It is not guaranteed to be the first visible prose token.

A7 / Post-prefill bound and acceptance

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Upper bound median (ms)			Upper bound P95 (ms)			Draft acceptance (%)		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	3.515	3.874	6.039	4.612	7.540	7.410	71.44	71.41	75.73
2	4.768	4.979	3.588	6.518	6.943	6.777	77.10	77.27	79.77
3	3.751	3.950	4.025	7.298	8.372	6.529	76.37	75.35	77.34
4	4.052	4.300	3.671	6.609	7.699	8.286	73.07	73.57	81.52
5	5.309	3.813	4.169	7.173	5.590	8.062	77.68	77.24	82.32
6	5.181	3.748	4.144	6.860	6.913	6.254	75.40	78.58	80.21
7	3.632	3.957	3.664	5.616	5.705	6.854	79.05	80.39	85.12
8	4.014	3.637	3.660	7.534	6.027	6.213	78.34	78.03	83.66
9	3.840	4.070	3.396	7.088	8.261	7.219	78.92	77.05	84.67
10	4.761	3.892	3.328	7.630	7.758	6.374	80.55	76.19	86.26
11	3.863	4.401	4.076	8.497	7.327	7.983	83.65	80.16	85.66
12	4.149	3.834	3.847	6.672	7.775	6.700	82.38	78.06	84.24

The first two columns are derived upper bounds, not exact pure-decode first-token times. Acceptance = accepted / drafted candidates in native counters. It is not task correctness or generic answer accuracy.

A8 / Calls, generated work and rejections

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Model requests			Generated tokens			Rejected tool attempts		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	5	5	5	2,956	2,956	2,956	1	1	1
2	8	8	8	3,602	3,602	3,602	0	0	0
3	12	15	14	5,602	6,252	8,286	0	2	2
4	18	19	19	8,942	8,651	8,981	1	2	2
5	25	20	23	9,617	9,268	10,789	2	0	2
6	27	25	26	13,201	11,071	13,772	2	1	2
7	29	28	29	11,966	10,975	11,465	1	0	1
8	33	33	34	15,847	16,075	13,820	1	1	2
9	41	43	37	21,467	22,845	18,975	4	6	0
10	49	49	40	23,500	27,159	17,163	9	9	0
11	51	51	46	22,694	22,980	21,206	5	5	2
12	52	54	50	24,845	25,688	22,244	4	4	2

Generated work includes reasoning. Each initial input is matched across profiles, but later prompts and request counts differ because the agent can choose a different tool trajectory. Rejections are intermediate and repaired.

A9 / Native draft and round counters

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Drafted candidates			Accepted candidates			Request-round sum		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	2,941	2,942	2,732	2,101	2,101	2,069	854	854	886
2	3,297	3,295	3,105	2,542	2,546	2,477	1,057	1,053	1,122
3	5,158	5,796	7,137	3,939	4,367	5,520	1,660	1,881	2,763
4	8,271	8,010	7,360	6,044	5,893	6,000	2,892	2,752	2,974
5	8,585	8,257	8,745	6,669	6,378	7,199	2,941	2,882	3,581
6	11,794	9,780	10,897	8,893	7,685	8,740	4,300	3,379	5,025
7	10,349	9,409	9,111	8,181	7,564	7,755	3,776	3,401	3,701
8	13,562	13,992	10,602	10,624	10,918	8,870	5,214	5,148	4,938
9	18,029	19,980	14,575	14,229	15,394	12,340	7,222	7,436	6,617
10	19,234	23,932	12,758	15,493	18,234	11,005	7,988	8,911	6,140
11	17,200	19,220	16,187	14,388	15,406	13,865	8,290	7,561	7,324
12	18,663	21,826	16,766	15,375	17,037	14,123	9,451	8,636	8,106

Request-round sum adds decode rounds of finished requests. It is not the count of unique shared GPU verification rounds, and must not be used as that denominator for shared-round throughput.

A10 / Prefix reuse and request-round output

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Prefix token queries			Prefix token hits			Output tokens/req-round		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	12,985	12,985	12,985	11,384	11,384	11,384	3.461	3.461	3.336
2	16,169	16,169	16,169	13,040	13,040	13,040	3.408	3.421	3.210
3	23,062	35,955	32,326	18,388	31,038	27,608	3.375	3.324	2.999
4	38,731	41,856	40,973	32,260	35,406	34,446	3.092	3.144	3.020
5	55,687	37,518	52,071	47,226	29,730	43,994	3.270	3.216	3.013
6	56,678	48,671	52,674	47,072	39,320	43,268	3.070	3.276	2.741
7	52,618	49,791	53,676	41,722	38,916	42,752	3.169	3.227	3.098
8	63,626	63,575	65,018	51,160	51,104	52,544	3.039	3.123	2.799
9	89,999	100,258	73,675	75,618	85,804	59,370	2.972	3.072	2.868
10	111,157	119,336	74,304	95,152	103,346	58,702	2.942	3.048	2.795
11	111,714	111,295	91,030	93,756	93,340	73,840	2.738	3.039	2.895
12	106,419	112,299	96,817	87,658	93,416	78,110	2.629	2.975	2.744

Prefix queries and hits are token-denominated counters, not numbers of snapshot objects. Output/request-round is generated tokens divided by summed finished-request rounds; it is not a shared-round device metric.

A11 / Summed native time and token reuse

Every cell is shown. Within each metric, columns are TF / Bert / xTF; no best-run selection.

C	Prefill seconds (sum)			Request decode s (sum)			Prefix token reuse (%)		
	TF	Bert	xTF	TF	Bert	xTF	TF	Bert	xTF
1	2.139	2.197	2.142	35.03	35.89	34.80	87.67	87.67	87.67
2	3.533	3.724	3.532	63.58	65.34	63.90	80.65	80.65	80.65
3	5.272	6.153	5.514	131.42	139.21	177.63	79.73	86.32	85.40
4	7.661	7.861	7.865	271.52	262.77	260.58	83.29	84.59	84.07
5	10.360	8.973	9.514	347.60	345.56	352.21	84.81	79.24	84.49
6	11.471	10.784	10.931	559.18	436.36	525.08	83.05	80.79	82.14
7	12.518	12.511	12.577	552.41	531.39	502.91	79.29	78.16	79.65
8	14.445	14.422	14.445	811.52	806.64	697.44	80.41	80.38	80.81
9	16.887	17.290	16.558	1,152.55	1,208.10	1,076.89	84.02	85.58	80.58
10	19.221	19.548	17.800	1,295.45	1,372.24	1,040.32	85.60	86.60	79.00
11	21.446	21.464	19.950	1,498.39	1,521.84	1,345.48	83.93	83.87	81.12
12	21.816	22.355	21.635	1,627.88	1,688.44	1,554.69	82.37	83.19	80.68

Request decode seconds overlap under concurrency, so their sum can exceed wall-clock wave duration. Token reuse is hits / queries. Native generated/computed token and request counts match A5/A8; complete counters are supplied as CSV.

A12 / Exact error strings and coverage

All errors in the measurement evidence are retained below; no error category was silently dropped.

Tool error returned to the model	1.2.2TF	Bert1	1.2.2xTF
unsupported pure-function syntax: Import	10	8	7
unsupported pure-function syntax: ImportFrom	1	3	3
one merge_events function required	1	2	1
unknown tool or invalid arguments	11	10	2
incorrect accounting: requests	6	6	2
provide a concise Czech explanation, 30 to180 words	0	1	1
provide four to six explanatory sentences	1	1	0

Complete runtime coverage

All 36 cells passed: correct = total = C. Runtime timing coverage equals model request count in every cell (350 TF, 350 Bert1, 331 xTF). coverage=True and failures_retained=True throughout. No measured native cancellation or preemption occurred.

The CSV keeps every original comparison field, including these constant coverage flags and rate_scope. Per-task results (234 rows) and per-request timings (1,031 rows) are supplied without conversations, code outputs or infrastructure identifiers.

Reading the results without mixing the metrics

A compact guide for comparing local inference recipes in your own setup.

Term	Meaning in this report
C / active streams	C is concurrent workflow tasks; active native streams vary as tools and requests interleave.
R / verification rows	Total shared-round rows, including mandatory rows; not a fixed draft length per stream.
KV pool / snapshot cache	Runtime context capacity / reusable prompt-state budget. Separate capacities and mechanisms.
Native decode / workflow TPS	Generation-stage per-request speed / tokens over full request or wave wall time.
P95 / pooled median	Descriptive observed quantile / all requests or tasks combined. No confidence interval is implied.
Acceptance / task pass rate	Draft agreement with target / final workflow correctness. Different measurements.

Practical takeaway: xTF was the strongest overall recipe in this pass, with better C5–C8 native decode and fewer repair attempts. Keep task duration, output volume and error categories beside TPS when judging a change.

Sources, attribution and reproducibility

Own measurements are distinct from research context; paper speedups are not reported as our results.

[1] Leviathan et al. (2023), Fast Inference from Transformers via Speculative Decoding [open source]

Background: target-distribution-preserving speculative inference. This benchmark does not independently prove every numerical implementation property.

[2] Liu et al. (2026), AdaSpark: Adaptive DSpark with Online Learning for Tree Verification and N-gram Fill [open source]

Inspiration for online cost/acceptance feedback and average-reward pricing. xTF is an adaptation in existing chain/copy lanes, not a full AdaSpark or imparo reproduction.

[3] PyTorch documentation, Numerical accuracy [open source]

General background on non-bit-identical floating-point results across execution shapes; not a diagnosis of this TensorFold runtime.

[4] vLLM documentation, Automatic Prefix Caching [open source]

Background on reusing prefill work. The engine tested here is TensorFold, not vLLM.

Own evidence: 36 frozen benchmark cells; numeric comparison and native counters; learner counters; exact tool-error breakdown; frozen source audit of adaptive.zig and its integration. Pre-benchmark preparation labels are historical; the measured GPU results in this report establish the later workflow run.

Companions: all-metric CSV, native counters, per-task and per-request CSV, summary JSON, source SHA-256 manifest, and SVG/PNG figures. Input hashes were rechecked when building this report. The share bundle contains numeric evidence and figures, with private deployment data omitted.